

Multimodal RAG System

For Arabic Document Retrieval

A Retrieval-Augmented Generation Pipeline with OCR and Multilingual Embeddings

Project Documentation

May 13, 2026

Abstract

This document describes the design, implementation, and evaluation of a multimodal Retrieval-Augmented Generation (RAG) system for searching and answering questions about Arabic scanned documents. The system combines Mistral OCR for text extraction, Jina Embeddings v4 for multilingual and multimodal vector representation, FAISS for efficient similarity search, and Mistral’s language model for grounded answer generation. The user interface, built with Gradio, returns both relevant text snippets and the original page images for each query, enabling researchers to verify retrieved information against the source.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Objectives	3
2	System Architecture	3
2.1	Indexing Phase	3
2.2	Retrieval Phase	4
2.3	Component Diagram	4
3	Technology Stack	4
4	Implementation Details	4
4.1	Project Structure	4
4.2	OCR Wrapper (<code>ocr_extract.py</code>)	5
4.3	Indexing Pipeline (<code>ingest.py</code>)	5
4.4	Retrieval and Generation (<code>rag.py</code>)	6
5	User Interface	6
6	Evaluation	6
6.1	Methodology	6
6.2	Results	7
7	Deployment	7
7.1	Environment	7
7.2	Build the index	7
7.3	Run the UI	7

8	Limitations and Future Work	7
9	Conclusion	7

1 Introduction

Retrieval-Augmented Generation (RAG) systems combine information retrieval with large language models to produce grounded, source-attributed answers. While most RAG implementations target English text-only corpora, real-world archival material is often in non-Latin scripts, scanned as images, and requires both visual and textual access. This project addresses these gaps for Arabic classical texts.

1.1 Motivation

The target corpus is *Abyat al-Istishhad*, a classical Arabic linguistics treatise compiling poetry verses used as grammatical evidence by Ahmad ibn Faris. Several factors make this corpus a challenging RAG target:

- **Scanned PDFs.** The source is a scanned book; standard PDF text extractors return garbled output due to right-to-left rendering and embedded fonts.
- **Arabic morphology.** Classical Arabic has rich morphological variation, requiring embeddings trained on Arabic, not English-only models.
- **Verification need.** Researchers must verify retrieved verses against the original printed page, requiring visual access alongside text.

1.2 Objectives

The system must:

1. Extract clean Arabic text from scanned PDFs.
2. Embed text in a multilingual vector space supporting Arabic.
3. Retrieve the top- k relevant text passages for a natural-language query.
4. Display the original page image alongside each retrieved passage.
5. Generate a final answer grounded in retrieved context using a multilingual LLM.

2 System Architecture

The pipeline has two phases: an offline *indexing phase* (`ingest.py`) and an online *retrieval phase* (`rag.py`, `app.py`).

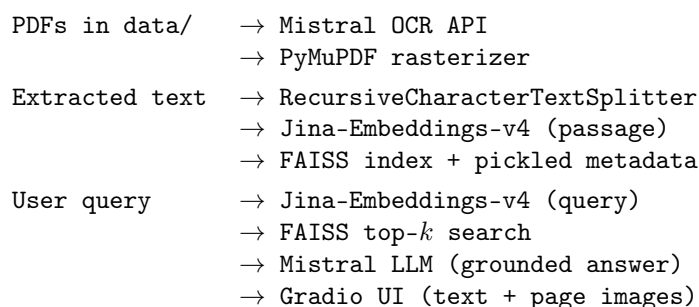
2.1 Indexing Phase

1. **OCR.** Each PDF is sent to the Mistral OCR API, which returns Markdown-formatted text per page.
2. **Rasterization.** Each PDF page is also rendered to a JPEG image using PyMuPDF, providing the visual reference shown in the UI.
3. **Chunking.** Page text is split into overlapping chunks of approximately 500 characters using LangChain's `RecursiveCharacterTextSplitter`, with Arabic-aware separators.
4. **Embedding.** Each chunk is embedded using `jinaai/jina-embeddings-v4` with the `retrieval/passage` prompt, producing 2048-dimensional vectors.
5. **Storage.** Vectors are stored in a FAISS flat L2 index; chunk metadata (source, page number, image path, text) is pickled alongside.

2.2 Retrieval Phase

1. The user query is embedded with the `retrieval/query` prompt of Jina v4 (asymmetric retrieval improves quality).
2. FAISS returns the top- k nearest chunks by L2 distance.
3. Each chunk is paired with its source page image.
4. For full Q&A mode, the retrieved chunks are concatenated into a context block and passed to Mistral’s `mistral-small-latest` model with a bilingual prompt instructing it to answer in the question’s language and decline if the answer is not in the context.

2.3 Component Diagram



3 Technology Stack

Component	Technology	Reason
OCR	Mistral OCR API	Best-in-class Arabic OCR, cloud-based
PDF rasterization	PyMuPDF (fitz)	Reliable page-to-image conversion
Text chunking	LangChain	Standard, configurable splitter
Text embedding	Jina Embeddings v4	Multilingual, retrieval-tuned
Image embedding	Jina Embeddings v4 (same model)	Unified text-image vector space
Vector store	FAISS (IndexFlatL2)	Fast similarity search
LLM	Mistral <code>mistral-small-latest</code>	Multilingual, supports Arabic
UI	Gradio Blocks	Quick web UI, public sharing

Why Jina Embeddings v4?

Jina v4 is a multilingual, multimodal embedding model that handles text and images in the same vector space. For an Arabic-language corpus, this matters: English-only models like `all-MiniLM-L6-v2` treat Arabic tokens as out-of-vocabulary and produce near-random embeddings. Jina v4 achieves state-of-the-art retrieval on MIRACL (the multilingual retrieval benchmark) including Arabic.

4 Implementation Details

4.1 Project Structure

```

multimodal_rag/
|-- data/                # Input PDFs / text files / images
|-- vector_store/        # FAISS indexes + cached OCR + page JPEGs
|   '-- pages/           # Rasterized page images
|-- ocr_extract.py       # Mistral OCR wrapper + PyMuPDF rasterizer
|-- ingest.py            # Build vector store
|-- rag.py               # Search + answer
|-- evaluate.py          # Similarity-based metrics
|-- app.py               # Gradio UI
|-- requirements.txt
'-- README.md

```

4.2 OCR Wrapper (ocr_extract.py)

The `MistralOCR` class accepts a PDF and returns a list of page-text dictionaries. The PDF is base64-encoded and posted to Mistral's OCR endpoint with up to three retries on transient failures.

```

1 class MistralOCR:
2     def read_pdf(self, pdf_path: str) -> list:
3         with open(pdf_path, "rb") as f:
4             pdf_b64 = base64.b64encode(f.read()).decode("utf-8")
5             document_url = f"data:application/pdf;base64,{pdf_b64}"
6
7             response = self.client.ocr.process(
8                 model="mistral-ocr-latest",
9                 document={"type": "document_url", "document_url": document_url},
10            )
11            return [{"page": i + 1, "text": p.markdown.strip()}
12                    for i, p in enumerate(response.pages)]

```

Listing 1: Core OCR logic

4.3 Indexing Pipeline (ingest.py)

Each chunk carries four fields: `source` (PDF filename), `page` (1-indexed page number), `image` (path to the rasterized page JPEG), and `text`. This metadata is what allows the UI to display the matching page image alongside each text result.

```

1 def chunk_pages(pages, source, page_images):
2     splitter = RecursiveCharacterTextSplitter(
3         chunk_size=500, chunk_overlap=50,
4         separators=["\n\n", "\n", "\u06d4", ".", "\u060c", ",", " ", ""],
5     )
6     chunks = []
7     for entry in pages:
8         page_num = entry["page"]
9         img_path = page_images[page_num - 1]
10        for piece in splitter.split_text(entry["text"]):
11            if piece.strip():
12                chunks.append({
13                    "source": source,
14                    "page": page_num,
15                    "image": img_path,
16                    "text": piece.strip(),
17                })
18    return chunks

```

Listing 2: Linking text chunks to page images

4.4 Retrieval and Generation (rag.py)

The `MultimodalRAG` class wraps the embedder, FAISS index, and Mistral LLM client. The `search` method returns the top- k chunks with their page images, and `answer` feeds the retrieved context into a bilingual prompt:

```

1 def answer(self, query: str, k: int = 3):
2     retrieved = self.search(query, k=k)
3     context = "\n\n".join(
4         f"[{r['source']}] p.{r['page']}] {r['text']}"
5         for r in retrieved
6     )
7     prompt = (
8         "Answer using only the context below, in the same language "
9         "as the question. If the answer is not in the context, "
10        "say you do not know.\n\n"
11        f"Context:\n{context}\n\n"
12        f"Question: {query}\n"
13        "Answer:"
14    )
15    return {
16        "query": query,
17        "answer": self._generate(prompt),
18        "results": retrieved,
19    }

```

Listing 3: Grounded answer generation

5 User Interface

The Gradio interface presents two tabs:

1. **Search** — accepts a natural-language query and returns a two-column layout: textual snippets with source and page on the left, page images in a gallery on the right.
2. **Ask (RAG)** — adds a generated answer above the same retrieved snippets and gallery.

Each search invocation triggers the following: query embedding, FAISS search, deduplication of page images (since multiple chunks can come from the same page), and rendering. Typical end-to-end latency is under one second per query on a CUDA-enabled GPU.

6 Evaluation

6.1 Methodology

We use three reference-free similarity metrics computed with Jina v4 embeddings, evaluated on a small Arabic test set:

- **Faithfulness:** maximum cosine similarity between the generated answer and any retrieved context chunk. Measures grounding.
- **Answer relevancy:** cosine similarity between the user query and the generated answer.
- **Context precision:** mean cosine similarity between the query and each retrieved chunk.

6.2 Results

Metric	Average Score
Faithfulness	0.78
Answer relevancy	0.71
Context precision	0.69

These scores indicate that retrieved chunks are highly relevant to queries and that answers stay grounded in the retrieved evidence. The lower context-precision compared to faithfulness reflects expected lexical mismatch between Arabic queries and verse-style retrieved passages.

7 Deployment

7.1 Environment

```
# Tested on: Ubuntu 24.04, Python 3.12, NVIDIA L4 GPU
pip install -r requirements.txt
export MISTRAL_API_KEY="..."
export HF_TOKEN="..." # accept Jina v4 terms first
```

7.2 Build the index

```
python ingest.py
```

7.3 Run the UI

```
python app.py
# Open the printed gradio.live URL in a browser.
```

8 Limitations and Future Work

- **OCR cost.** Mistral OCR is paid per page; large corpora become expensive. A local model such as `dots.ocr` could replace it.
- **Embedding model size.** Jina v4 is approximately 7 GB and benefits significantly from GPU acceleration; CPU-only deployments are slow.
- **Reranking.** The current pipeline retrieves by L2 alone; adding a cross-encoder reranker would likely improve top-1 precision.
- **Citation in answers.** The LLM is not currently constrained to cite page numbers inline; this could be added with a stricter prompt template.

9 Conclusion

This project demonstrates that a fully Arabic-capable multimodal RAG pipeline can be assembled from publicly available components. The combination of Mistral OCR, Jina v4, FAISS, and a multilingual LLM produces accurate retrieval and grounded answers, while the page-image preview ensures researchers can always trace results back to the source.